

Before beginning

- Our primary goal in these lectures is learning how to produce software that is:
 - ✓ **Safe from bugs**
 - ✓ **Easy to understand**
 - ✓ **Ready for change**

1-Specifications

September 18, 2024



Lecture Overview

1. Introduction
2. Behavioral Equivalence
3. Specification Structure
4. Specifications in Java
5. Specification for a Mutating Method
6. Two kinds of Specifications
7. Judging Specifications
8. Exceptions

Introduction

- The specification acts as a **contract**:
 - The implementer is responsible for meeting the contract, and a client that uses the method can rely on the contract.
- It's impossible to delegate responsibility for implementing a method without a specification.
- Specifications place demands **on both parties**.

Introduction

- Many of the nastiest bugs in programs arise because of **misunderstandings** about behavior at interfaces.
- Specifications are **good for the client** of a method because they spare the task of reading code
- Specifications are **good for the implementer** of a method because they give the freedom to change the implementation without telling clients.

Introduction

- The contract acts as *a firewall between client and implementor*.
 - It shields the client from the details of the *workings of the unit*.
 - And it shields the implementor from the details of the *usage of the unit*.
- This firewall results in *decoupling*, allowing the *code* of the unit and the code of a client to be changed independently, so long as the changes respect the specification.

Behavioral Equivalence

- An example
- whether one could substitute one implementation for the other

```
public static int findA (int[] a, int val) {  
    for (int i = 0; i < a.length; ++i) {  
        if (a[i] == val) {  
            return i;  
        }  
    }  
    return a.length;  
}
```

```
public static int findB (int[] a, int val) {  
    for (int i = a.length-1; i >= 0; --i) {  
        if (a[i] == val) {  
            return i;  
        }  
    }  
    return -1;  
}
```

Behavioral Equivalence

- In order to make it possible to substitute one implementation for another, and to know when this is acceptable, we need a specification that states exactly what the client depends on.
- For example, give a specification

Requires: val occurs in a

Effects: returns result such that $a[\text{result}] = \text{val}$

Specification Structure

- A specification of a method consists of several clauses:
 - a **precondition**, indicated by the keyword *requires*;
 - a **postcondition**, indicated by the keyword *effects*;
 - a **frame condition**, indicated by the keyword *modifies*.

Specification Structure

- The precondition is an obligation on the client.
- The postcondition is an obligation on the implementer of the method.
- The frame condition identifies which objects may be modified.
- If you omit the frame condition, the default is *modifies nothing. In other words, the method makes no changes to any object.*

Specifications in Java

- **Static type declarations** *are effectively part of the precondition and postcondition of a method, a part that is automatically checked and enforced by the compiler.*
- The rest of the contract – the parts that we can't write as types – must be described in a comment preceding the method, and generally depends on human beings to check it and guarantee it.

Specifications in Java

■ Example

Static int find (int[] a, int val)

Requires: val occurs exactly once in a

Effects: returns result such that a[result] = val

```
/**  
 * Find value in an array  
 * @param a array to search; requires that val occurs exactly once in a.  
 * @param val value to search for  
 * @return index i such that a[i] = val  
 */  
static int findA (int[] a, int val)
```

Specifications in Java

- About deterministic and under-determined specification

Static int find (int[] a, int val)

Requires: val occurs exactly once in a

Effects: returns result such that $a[\text{result}] = \text{val}$

Static int find (int[] a, int val)

Requires: val occurs in a

Effects: returns result such that $a[\text{result}] = \text{val}$

Specifications in Java

- Specification that distinguishes the two methods.

```
/**  
 * Find value in an array  
 * @param a a array to search.  
 * @param val value to search for.  
 * @return returns largest result such that a[result] = val or -1 if no such  
result.  
 */
```

Specification for a Mutating Method

- Here's a specification that describes a method that mutates an object: (Java List interface)

static boolean addAll (List l1, List l2)

requires: l1 != l2, l1 != null, l2 != null

modifies: this

effects: adds the elements of l2 to the end of l1, and returns true if l1 changed as a result of call

Two kinds of Specifications

- Roughly speaking, there are two kinds of specifications.
 - *Operational* specifications
 - *Declarative* specifications
- Almost always, **declarative specifications are preferable**. They're usually shorter, easier to understand, and most importantly, they don't expose implementation details inadvertently that a client may rely on

Two kinds of Specifications

- Suppose you want to substitute one method for another. How do you compare the specifications?
- A specification **A is at least as strong as a specification B** if
 - A's precondition is no stronger than B's
 - A's postcondition is no weaker than B's, for the states that satisfy B's precondition.

Judging Specifications

- What makes a good method?
 - Designing a method means primarily writing a specification
- About the form of the specification:
 - it should obviously be succinct,
 - clear
 - well-structured.
- The content is harder to prescribe.

Judging Specifications

- Some useful guidelines:
 - It should be *coherent*

```
static int minFind (int[] a, int[] b, int val)
```

effects: returns smallest index in arrays a and b at which val appears

Is this a well-designed procedure?

Probably not: it's incoherent, since it does two things (finding and minimizing) that are not really related.

It would be better to use two separate procedures.

Judging Specifications

- Some useful guidelines:
 - The results of a call should be *informative*.

```
static V put (Map<K,V> map, K key, V val)
```

effects: inserts (key, val) into the mapping, overriding any existing mapping for key, and returns old value for key, unless none, in which case it returns null.

Note that the precondition does not rule out null values, so the map can store nulls. But the postcondition uses null as a special return value for a missing key.

This means that if null is returned, you can't tell whether the key was not bound previously, or whether it was in fact bound to null.

Judging Specifications

- Some useful guidelines:
 - The specification should be *strong enough*.

```
static void addAll (List<T> l1, List<T> l2)
```

effects: adds the elements of l2 to l1, unless it encounters a null element, at which point it throws a `NullPointerException`

There's no point throwing a checked exception for a bad argument but allowing arbitrary mutations, because a client won't be able to determine what mutations have actually been made.

Judging Specifications

- Some useful guidelines:
 - The specification should be *weak enough*

static File open (String filename)

effects: opens a file named filename.

This is a bad specification. It lacks important details: is the file opened for reading or writing? Does it already exist or is it created? And it's too strong, since there's no way it can guarantee to open a file.

Instead, the specification should say something much weaker: that it attempts to open a file, and if it succeeds, the file has certain properties.

Exceptions

- Exceptions for special results
 - A standard way to handle special results is to return special values.
 - Lookup operations in the Java library are often designed like this: you get an index of -1 when expecting a positive integer, or a null reference when expecting an object. (eg. `java.util.Arrays.binarySearch()`, `java.util.ArrayList.indexOf()`)
 - This approach is OK if used sparingly.
 - It has two problems though. First, it's tedious to check the return value. Second, it's easy to forget to do it. (eg. `java.util.ArrayList.get()` `remove()`)

Exceptions

- Exceptions for special results
 - Sometime, its not easy to find a “special value”.
 - Suppose we have a BirthdayBook class with a lookup method. Here’s one possible method signature:

Calendar lookup (String name)

- What should the method do if the birthday book doesn’t have an entry for the person whose name is given?

Exceptions

- Exceptions for special results
 - Here's a better approach. The method throws an exception:

```
Calendar lookup (String name) throws NotFoundException {  
    ...  
    if // not found  
        throw new NotFoundException ();  
    ...  
}
```

Exceptions

■ Abuse of Exceptions

```
try{  
    while(true){  
        if(a[i++] == val)  
            return i;  
    }  
}catch (ArrayIndexOutOfBoundsException e){...}
```

```
for (int i = 0; i < a.length; ++i) {  
    if (a[i] == val)  
        return i;  
}
```

Exceptions

- Checked and Unchecked Exceptions
 - We've seen two different purposes for exceptions: failures and special results.
 - Java provides two different kinds of exception for these two purposes.
 - They behave the same at runtime; the only difference is what kind of checking the compiler provides.
 - *checked* exception
 - *unchecked* exception

Exceptions

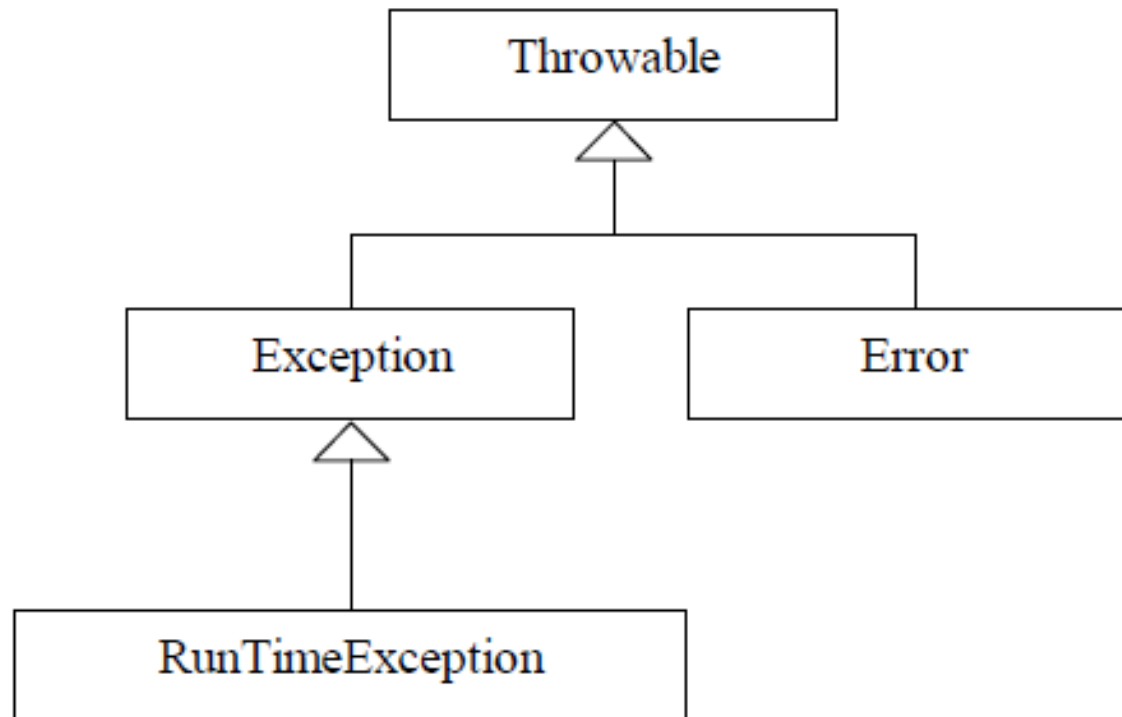
- Exceptions and Preconditions
 - An obvious design issue is whether to use a precondition, and if so, whether it should be checked.
 - Sometimes, it's not feasible to check a condition without making a method unacceptably slow, and a precondition is often necessary in this case.
 - The decision of whether to use a precondition is an engineering judgment.
 - The key factors are the cost of the check and the scope of the method.

Exceptions

- Design Considerations
 - You should use an unchecked exception only if you expect that clients will usually write code that ensures the exception will not happen, because there is a convenient and inexpensive way to avoid the exception, or because the exception reflects unexpected failures;
 - Otherwise you should use a checked exception

Exceptions

- Let's look at the class hierarchy for Java exceptions.



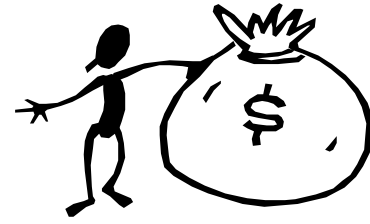
Exercises

- Hailstone

- 给定任意自然数 n ，求解 $H(n)$ 序列中的峰值。
- 例如 $H(3)=\{3, 10, 5, 16, 8, 4, 2, 1\}$ ，峰值为16

$$H(n) = \begin{cases} 1 & n = 1 \\ n / 2 & n \% 2 = 0 \\ 3n + 1 & n \% 2 \neq 0 \end{cases}$$

Summary



Discuss

